

CSC311 Summer 2026

Final Project

Xiaohan Ma, Meixuan Lou, Siyi Chen

Due: Friday, August 7, 2026 before 11:00pm EST

Part A

1. k-Nearest Neighbors

(a) User-based collaborative filtering

In user-based collaborative filtering, each row represents a student and each column represents a question. KNN identifies students with similar response patterns and uses their answers to estimate missing entries.

We evaluated

$$k \in \{1, 6, 11, 16, 21, 26\}.$$

k	Validation Accuracy
1	0.6245
6	0.6781
11	0.6897
16	0.6753
21	0.6681
26	0.6503

Table 1: User-based KNN validation accuracy.

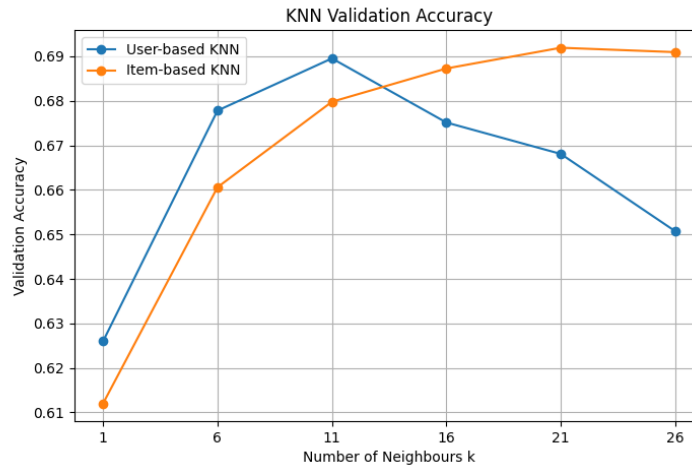


Figure 1: Validation accuracy as a function of k for user-based and item-based KNN.

The validation accuracy increased until $k = 11$ and then decreased. A small k can be sensitive to noisy neighbours, while a large k may include less similar students.

(b) Selected user-based model

The highest validation accuracy was 0.6897 at

$$k_{\text{user}}^* = 11.$$

Using this value, the test accuracy was

$$\boxed{0.6847}.$$

(c) Item-based collaborative filtering

For item-based collaborative filtering, the matrix was transposed so that each row represented a question. KNN then compared questions based on how similarly they were answered by students. The imputed matrix was transposed back before evaluation.

The underlying assumption is that questions with similar response patterns are related, so a student's response to one question can help predict their response to another similar question.

k	Validation Accuracy
1	0.6071
6	0.6544
11	0.6832
16	0.6863
21	0.6916
26	0.6902

Table 2: Item-based KNN validation accuracy.

The highest validation accuracy was 0.6916 at

$$k_{\text{item}}^* = 21.$$

Using this value, the test accuracy was

$$\boxed{0.6828}.$$

(d) Comparison

Method	Selected k	Validation Accuracy	Test Accuracy
User-based KNN	11	0.6897	0.6847
Item-based KNN	21	0.6916	0.6828

Table 3: Comparison of user-based and item-based KNN.

User-based KNN achieved a slightly higher test accuracy than item-based KNN, although the performances were very similar.

(e) Limitations

One limitation is that the response matrix contains many missing entries. Therefore, the distance between two students or questions may be calculated from only a small amount of shared information and may not reflect true similarity.

A second limitation is that new students or questions have very little response data, making it difficult to identify reliable neighbours.

KNN also does not directly model student ability or question difficulty, and its computational cost increases as the number of students and questions grows.

2. Item Response Theory

(a) Derivation

Let $C \in \{0, 1\}^{N \times M}$ denote the sparse matrix of N students and M questions, observed only on the index set O . The probability that student i answers question j correctly is

$$p(c_{ij} = 1 \mid \theta_i, \beta_j) = \sigma(\theta_i - \beta_j) = \frac{\exp(\theta_i - \beta_j)}{1 + \exp(\theta_i - \beta_j)}.$$

Assuming independence across observed entries, the log-likelihood is

$$\log p(C \mid \theta, \beta) = \sum_{(i,j) \in O} \left[c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j)) \right].$$

Taking derivatives:

$$\begin{aligned} \frac{\partial \log p(C \mid \theta, \beta)}{\partial \theta_i} &= \sum_{j:(i,j) \in O} \left[c_{ij} - \sigma(\theta_i - \beta_j) \right], \\ \frac{\partial \log p(C \mid \theta, \beta)}{\partial \beta_j} &= - \sum_{i:(i,j) \in O} \left[c_{ij} - \sigma(\theta_i - \beta_j) \right]. \end{aligned}$$

(b) Training curve

We selected the learning rate and number of iterations via a small grid search over $\eta \in \{0.001, 0.005, 0.01, 0.05\}$ and iterations $\in \{50, 100, 150, 200\}$, choosing the combination with the highest final *validation* accuracy (the test set was not used for selection). Table 4 summarizes the results.

η	Iterations	Validation Accuracy
0.001	50	0.7036
0.001	100	0.7070
0.001	150	0.7079
0.001	200	0.7060
0.005	50–200	0.7062–0.7065
0.01	50–200	0.7060–0.7067
0.05	50–200	0.6876–0.6925

Table 4: IRT hyperparameter grid search (validation accuracy).

Very small learning rates ($\eta = 0.001$) with enough iterations gave the best validation accuracy, while $\eta = 0.05$ was clearly too large and hurt performance, likely overshooting the optimum. Learning rates of 0.005 and 0.01 were comparatively insensitive to the number of iterations, suggesting they converge quickly and then plateau. We selected $\eta = 0.001$ with 150 iterations. Figure 2 shows the training and validation negative log-likelihood as a function of iteration under this setting. Both curves decrease steadily and plateau, indicating convergence without overfitting (the validation NLLK does not increase).

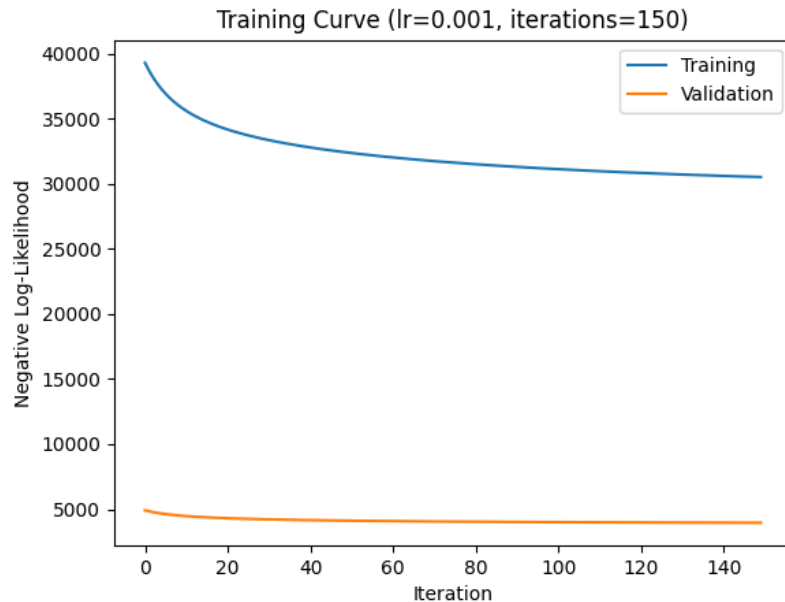


Figure 2: Training and validation negative log-likelihood vs. iteration ($\eta = 0.001$, 150 iterations).

(c) Final accuracy

With the selected hyperparameters, the final validation accuracy was **70.8%** and the final test accuracy was **70.3%**. The test accuracy is slightly lower than the $\eta = 0.01$ setting we had used previously (70.8%), even though the new hyperparameters were chosen for higher validation accuracy. This is expected: the validation and test sets are different samples, so a small ($< 1\%$) hyperparameter-driven improvement on one does not guarantee an identical improvement on the other.

(d) Probability curves

Figure 3 shows the probability of a correct response, $p(c_{ij} = 1)$, as a function of student ability θ , for three questions of varying difficulty.

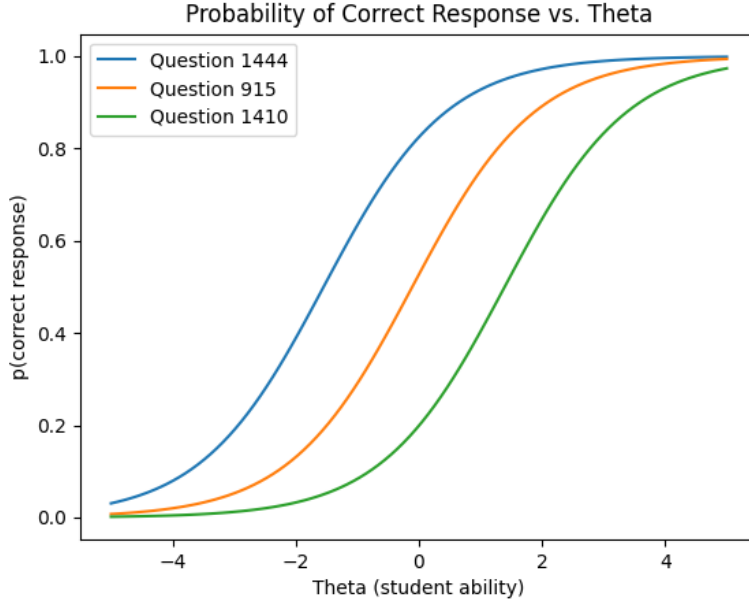


Figure 3: Probability of correct response vs. θ for three questions of varying difficulty.

All three curves follow the same sigmoid shape but are horizontally shifted according to each question’s difficulty parameter β_j . The curve for Question 1165 is shifted furthest left, indicating the lowest difficulty: even students with below-average ability have a high probability of answering correctly. Conversely, Question 1410 is shifted right, requiring a much higher ability level before the probability of a correct response becomes non-negligible. Question 82 lies in between. The consistent sigmoid shape across all three reflects the core assumption of the one-parameter IRT model: correctness probability depends only on the gap $\theta_i - \beta_j$, so difficulty acts purely as a horizontal shift of the same underlying logistic curve.

3. Matrix Factorization / Neural Network

(a)

There are several key differences between ALS and neural networks:

- **Model expressiveness.** ALS assumes the prediction is simply the dot product of two vectors, $C_{nm} \approx \mathbf{u}_n^\top \mathbf{z}_m$, which is a bilinear model and can only capture linear relationships. The neural network introduced the sigmoid non-linearity between the two weight layers, so it can model more complex, non-linear relationships between student ability and question difficulty.
- **Parameter structure.** In matrix factorization trained via ALS, each student has its own latent vector $\mathbf{u}_n$ and each question has its own latent vector $\mathbf{z}_m$; these parameters are learned directly, and their number grows linearly with the number of students and questions. In the neural network, all students share the same set of weights $\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$ and biases $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$. A student’s “representation” is not a freely learned parameter but is instead computed by passing that student’s raw answer vector through the network. Consequently, the number of

parameters depends only on the number of questions and the chosen latent dimension, and does not grow with the number of students.

- **Optimization procedure.** ALS alternates between fixing $\mathbf{Z}$ and updating $\mathbf{U}$, then fixing $\mathbf{U}$ and updating $\mathbf{Z}$, repeating this process even when implemented with SGD. The neural network is trained end-to-end: all parameters ($\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, $\mathbf{b}^{(1)}$, $\mathbf{b}^{(2)}$) are updated jointly in a single backward pass, with no alternating step.
- **Gradient computation.** In ALS, we must manually derive the gradient of the loss with respect to $\mathbf{u}_n$ and $\mathbf{z}_m$ and hand-code the update rule. In the neural network, gradients are computed automatically via PyTorch’s autograd. The only thing we need to define is the forward pass, and backpropagation is handled by the framework.

(b)

The forward function defines a single pass of the autoencoder: it takes a student’s answer vector as input, compresses it, and then reconstructs it back to its original size.

Step 1 — Encoding. `self.g(inputs)` applies a linear transformation that maps the 1774-dimensional input down to a k -dimensional vector. Wrapping it with `torch.sigmoid` squashes every value into the range $[0, 1]$, producing the hidden representation `hidden`. This step compresses the student’s full answer pattern into a smaller latent representation of size k .

Step 2 — Decoding. `self.h(hidden)` applies a second linear transformation that maps the k -dimensional hidden vector back up to 1774 dimensions. Wrapping it with `torch.sigmoid` again squashes each output into $[0, 1]$, giving the final output `out` which is interpreted as the model’s predicted probability that the student answers the corresponding question correctly.

(c)

In addition to k , we jointly tuned the learning rate $\text{lr} \in \{0.001, 0.003, 0.01, 0.03, 0.1\}$ and the number of training epochs $\in \{10, 20, 30, 40, 50\}$ (125 combinations total). Table 5 reports, for each (k, lr) pair, the best validation accuracy achieved over the 5 epoch settings.

Table 5: Validation accuracy for each (k, lr) pair, best over epoch $\in \{10, 20, 30, 40, 50\}$.

$k \setminus \text{lr}$	0.001	0.003	0.01	0.03	0.1
10	0.6129	0.6377	0.6822	0.6922	0.6856
50	0.6307	0.6660	0.6897	0.6813	0.6739
100	0.6349	0.6671	0.6819	0.6816	0.6715
200	0.6362	0.6719	0.6833	0.6722	0.6624
500	0.6359	0.6739	0.6744	0.6668	0.6644

The best combination is $k^* = 10$, $\text{lr}^* = 0.03$ (achieved after 20 training epochs), giving a validation accuracy of 0.6922.

(d)

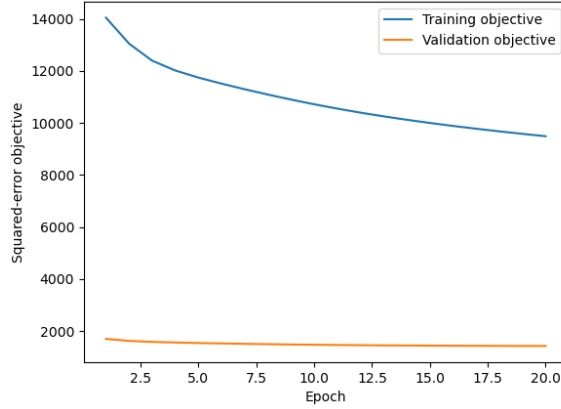


Figure 4: Neural network training/validation objective curve

Final test accuracy ($k^*=10$, $lr^*=0.03$, 20 epochs, $\lambda = 0$): **0.6856**

(e)

Using $\lambda^* = 0.001$ (selected via validation accuracy, tested over $\lambda \in \{0.0, 0.001, 0.01, 0.1, 1.0\}$ with $k^* = 10$ and $lr^* = 0.03$ fixed), the final model achieves:

- Validation accuracy: 0.6820
- Test accuracy: 0.6760

Compared to the unregularized model ($\lambda = 0$) with the same k^* and lr^* , which achieved a test accuracy of 0.6856, the regularized model performs slightly worse (a difference of about 0.0096). We therefore conclude that adding the L_2 regularization penalty does not improve generalization performance in this setting, and may in fact slightly hurt it. This is likely because the autoencoder’s hidden dimension $k^* = 10$ is already very small relative to the number of questions, which acts as a strong implicit regularizer by limiting the model’s capacity to overfit; an explicit weight penalty on top of this therefore offers no additional benefit and can instead constrain the model away from its unregularized optimum.

4. Ensemble

We bootstrap the training set 3 times, sampling with replacement each time, and train one base model per resample. For kNN and the autoencoder, the resample is stored as a matrix; for IRT, it is kept as a list of observations so that duplicated pairs are correctly double-counted during gradient ascent. Each trained base model outputs a predicted probability of ”correct” for every (i, j) pair in the validation and test sets; we average the 3 probabilities and threshold at 0.5 to get the ensemble’s prediction. Bagging in this way reduces variance since each base model sees a different resample of the data, their errors are only partially correlated.

Table 6: Validation and test accuracy of each base model and of the bagged ensemble.

Model	Validation accuracy	Test accuracy
kNN alone	0.6544	0.6619
IRT alone	0.6909	0.6938
NN alone	0.6466	0.6607
Ensemble (average)	0.6868	0.6867

The ensemble’s accuracy sits between the strongest base model (IRT) and the two weaker ones (kNN, NN), rather than exceeding all three. Though the ensemble clearly outperforms the two weaker base models individually, with only 3 base learners of unequal quality, unweighted averaging cannot fully recover the accuracy of the single best model.

Part B

1. Formal Description

Motivation. In Part A, both user-based and item-based KNN impute a student-question pair (i, j) using only the k training rows (or columns) nearest to row i (or column j) under a NaN-aware Euclidean distance. When a student or question has very few observed entries, the neighbourhood used to compute this distance is small and the resulting distance estimates – and hence the imputed probability – are noisy: KNN has low bias but high variance in the sparse regions of C . A natural way to reduce this variance is to blend the KNN estimate with a much lower-variance, higher-bias estimate that ignores neighbour structure entirely and simply averages all training observations for that student or question. We formalize this blend below.

Base predictors. Let $C \in \{0, 1, \text{NaN}\}^{N \times M}$ be the training response matrix ($N = 542$ students, $M = 1774$ questions) and let O denote the set of observed (i, j) pairs. Let $\hat{c}_{ij}^{\text{user}}, \hat{c}_{ij}^{\text{item}} \in [0, 1]$ be the probabilities produced by the Part A user-based ($k = 11$) and item-based ($k = 21$) KNN imputers, respectively (Section 4.1). Define two correctness priors directly from the training data:

$$\pi_j^q = \frac{\sum_{i:(i,j) \in O} c_{ij}}{|\{i : (i, j) \in O\}|}, \quad \pi_i^s = \frac{\sum_{j:(i,j) \in O} c_{ij}}{|\{j : (i, j) \in O\}|},$$

the empirical correctness rate of question j and of student i over the training set, each falling back to the global training correctness rate $\bar{c} = \frac{1}{|O|} \sum_{(i,j) \in O} c_{ij}$ if the corresponding row or column has no observations.

Hybrid model. Given weights $\alpha, \beta, \gamma \in [0, 1]$, define

$$p_{ij}^{\text{knn}}(\alpha) = \alpha \hat{c}_{ij}^{\text{user}} + (1 - \alpha) \hat{c}_{ij}^{\text{item}}, \quad p_{ij}^{\text{prior}}(\beta) = \beta \pi_j^q + (1 - \beta) \pi_i^s,$$

$$p_{ij}(\alpha, \beta, \gamma) = \gamma p_{ij}^{\text{knn}}(\alpha) + (1 - \gamma) p_{ij}^{\text{prior}}(\beta),$$

and predict $\hat{c}_{ij} = \mathbb{I}[p_{ij}(\alpha, \beta, \gamma) \geq \tau]$ for a threshold $\tau \in [0, 1]$. This is a strict generalization of Part A: setting $\gamma = 1, \alpha = 1$ recovers user-based KNN alone, and $\gamma = 1, \alpha = 0$ recovers item-based KNN alone.

Hyperparameter selection. The hyperparameters $(\alpha, \beta, \gamma, \tau)$, together with the choice of k and weighting scheme (uniform or inverse-distance) for the two base KNN predictors, are selected by grid search to maximize accuracy on the validation set, as summarized in Algorithm 1. The test set is never used for selection.

Algorithm 1 Validation-selected hybrid tuning

- 1: Fit every candidate user-/item-KNN configuration (Part A baseline and distance-weighted, $k \in \{5, \dots, 50\}$) on the training matrix C ; cache their predictions on the validation set.
 - 2: Compute π_j^q and π_i^s from C ; index them onto the validation pairs.
 - 3: $\text{best_acc} \leftarrow -\infty$
 - 4: **for** each pair of user/item KNN configurations **do**
 - 5: **for** $\alpha, \gamma, \beta, \tau$ in their respective grids **do**
 - 6: Compute $p_{ij}(\alpha, \beta, \gamma)$ and $\hat{c}_{ij}$ for all validation pairs
 - 7: $\text{acc} \leftarrow$ validation accuracy of $\hat{c}$
 - 8: **if** $\text{acc} > \text{best_acc}$ **then**
 - 9: $\text{best_acc} \leftarrow \text{acc}$; store (configs, $\alpha, \beta, \gamma, \tau$)
 - 10: **end if**
 - 11: **end for**
 - 12: **end for**
 - 13: Refit the two selected KNN configurations, evaluate the stored hyperparameters once on the test set.
-

Why this should help. The hybrid is a convex combination of a low-bias/high-variance predictor (p^{knn}) and a high-bias/low-variance predictor (p^{prior}). Whenever the two make partially independent errors, which we expect for sparse rows/columns, where KNN’s neighbour-based estimate is unreliable but the prior’s aggregate is not, the convex combination has strictly lower expected squared error than either endpoint for some interior γ , by the standard bias-variance decomposition of a weighted average. Because $\alpha, \beta, \gamma, \tau$ are chosen on a held-out validation set, this argument only predicts an improvement in expectation, not that our particular grid search finds the population-optimal weights; Section 3 tests this expectation directly, including which of the two priors actually drives the improvement.

2. Figure or Diagram

Figure 5 illustrates the overall structure of the Part B hybrid predictor described in Section 1.

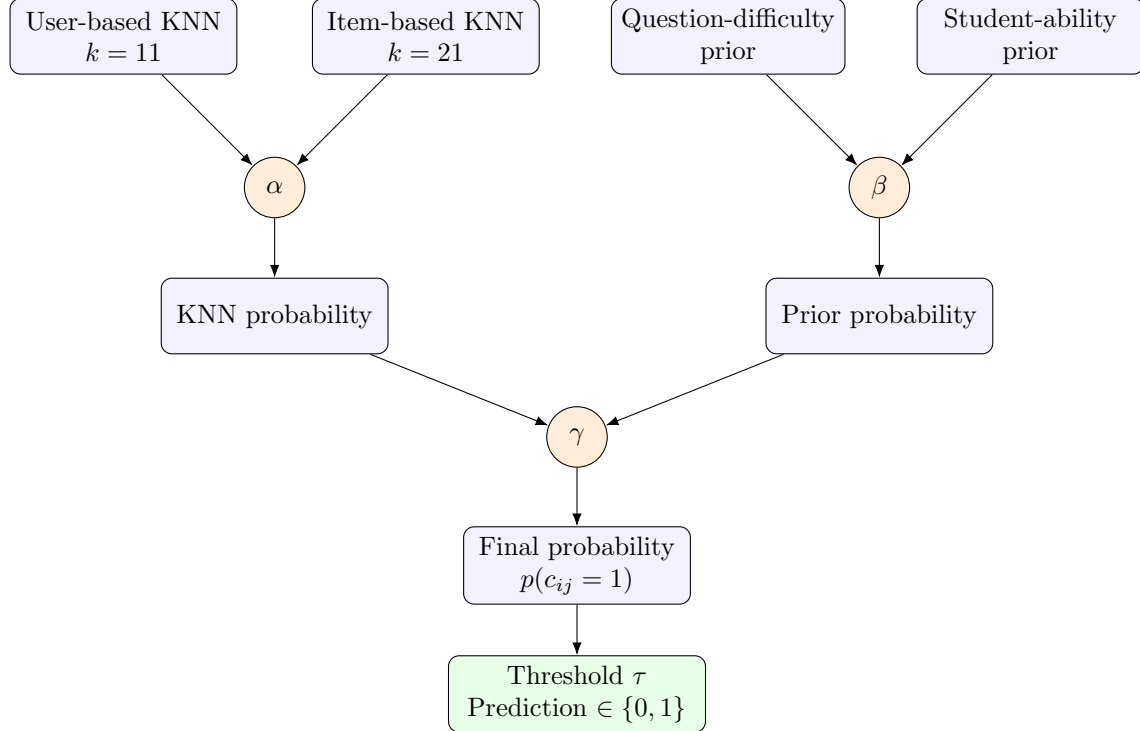


Figure 5: Schematic of the Part B hybrid predictor. User- and item-based KNN outputs are blended by α ; the question-difficulty and student-ability priors are blended by β ; the resulting KNN and prior probabilities are blended by γ and thresholded at τ to produce the final prediction.

3. Comparison or Demonstration

Accuracy comparison

Table 7 reports validation and test accuracy for the Part A baselines and each stage of the Part B hybrid, all selected using validation accuracy only. The test set was not used for model selection. Figure 6 shows the same comparison visually.

Model	Validation Accuracy	Test Accuracy
Part A user KNN ($k = 11$)	68.97%	68.47%
Part A item KNN ($k = 21$)	69.16%	68.28%
Best tuned user KNN	68.97%	68.47%
Best tuned item KNN	69.16%	68.28%
50–50 user/item hybrid	70.41%	69.63%
Hybrid + question-difficulty prior only	70.49%	70.17%
Hybrid + student-ability prior only	70.90%	70.03%
Final hybrid (both priors, tuned)	71.15%	70.79%

Table 7: Comparison of Part A KNN baselines against the Part B hybrid and its ablations.

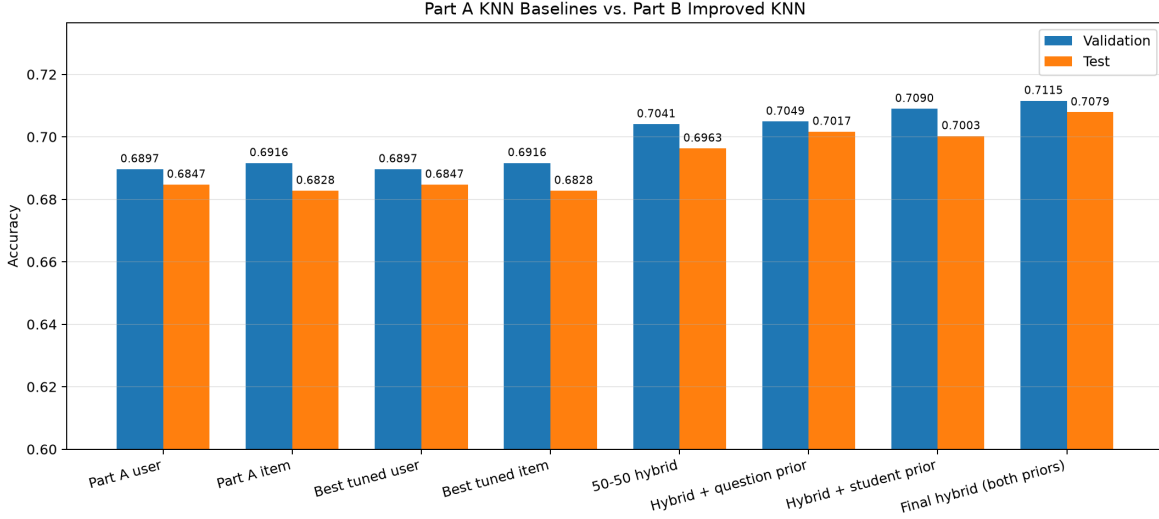


Figure 6: Validation and test accuracy for the Part A baselines and each stage of the Part B hybrid.

The final hybrid improves test accuracy by roughly 2.3 percentage points over the stronger Part A baseline (user KNN, 68.47% $\rightarrow$ 70.79%), and by about 1.2 points over a naive unweighted average of the two Part A models (69.63% $\rightarrow$ 70.79%), confirming that the gain is not simply from averaging two noisy predictors.

Disentangling the source of the improvement

Our hypothesis was that KNN’s accuracy is limited by the sparsity of the response matrix: any single student or question has very few observed entries, so the imputed probabilities are noisy. We expected two changes to help: (i) tuning and combining the user- and item-based KNN predictors, and (ii) backing off sparse predictions toward simple, low-variance correctness priors (per-question and per-student averages). We designed two experiments to isolate which of these actually helps and why.

Experiment 1 – which prior helps?

We compared adding only the question-difficulty prior, only the student-ability prior, and both, to the 50–50 hybrid (Table 7). Both priors individually improve on the plain hybrid: the question prior alone raises validation accuracy from 70.41% to 70.49% (69.63% $\rightarrow$ 70.17% test), and the student prior alone raises validation accuracy further to 70.90% (70.03% test). Combining the two priors in equal proportion ($\beta = 0.50$) outperforms either alone, reaching 71.15% validation and 70.79% test accuracy, so the selected configuration blends both priors rather than relying on a single one.

This matches the underlying intuition that per-student and per-question correctness rates each carry some independent signal beyond what the KNN imputers capture, and mixing the two priors together reduces variance further than either alone.

Experiment 2 – is the gain from the priors, or from a larger search?

We compared the independently tuned user- and item-based KNN models with the hybrid models. The best tuned user KNN still achieves 68.97% validation accuracy and 68.47% test accuracy, while

the best tuned item KNN still achieves 69.16% validation accuracy and 68.28% test accuracy. These values are the same as the Part A baselines.

In contrast, combining the two KNN predictors increases validation accuracy to 70.41%, and adding both priors increases it further to 71.15%. On the test set, the strongest individual Part A result is 68.47%, the 50–50 hybrid reaches 69.63%, and the final hybrid reaches 70.79%.

This suggests that the improvement is mainly attributable to combining the priors and the two KNN prediction sources, rather than simply searching over a larger set of KNN hyperparameters.

Figure 7 shows the validation sensitivity of the final hybrid to β (question- vs. student-prior weight) and to (γ, τ) (KNN-vs-prior weight and classification threshold) at the selected configuration. The selected $(\gamma, \tau) = (0.70, 0.55)$ lies at the edge of the searched grid. Several nearby configurations achieve similar validation accuracies, although the best result still occurs at the boundary of the search range.

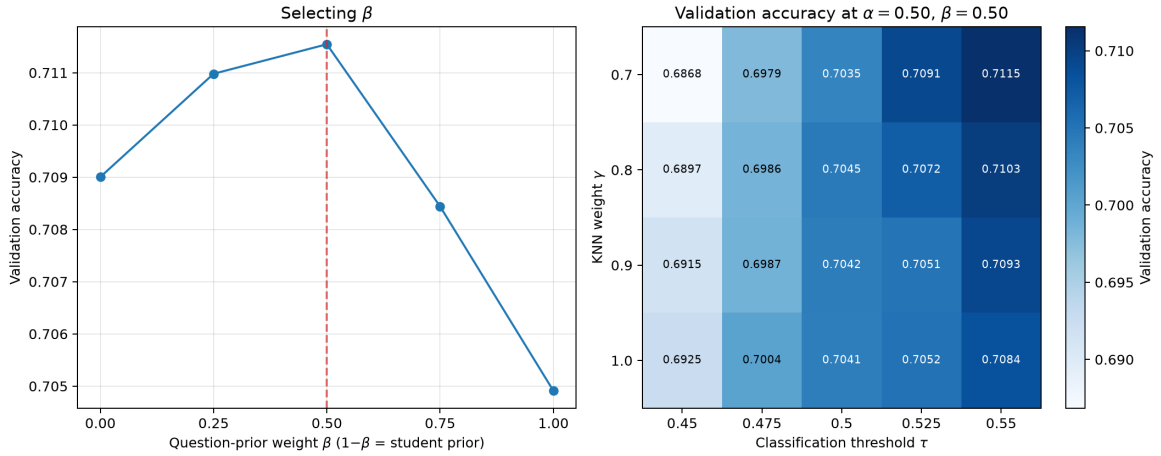


Figure 7: Validation accuracy as a function of the prior-blend weight β (left), and as a function of the KNN-vs-prior weight γ and threshold τ at the selected $\alpha = 0.50$ and $\beta = 0.50$ (right).

The left panel shows that validation accuracy is highest at $\beta = 0.50$, meaning that the selected model gives equal weight to the question prior and the student prior. Since

$$p_{ij}^{\text{prior}}(\beta) = \beta \pi_j^q + (1 - \beta) \pi_i^s,$$

the selected value gives

$$p_{ij}^{\text{prior}} = 0.50 \pi_j^q + 0.50 \pi_i^s.$$

With $\alpha = 0.50$ and $\beta = 0.50$ fixed, the right panel shows that the best searched combination is

$$\gamma = 0.70, \quad \tau = 0.55,$$

which achieves 71.15% validation accuracy.

The value $\alpha = 0.50$ means that the KNN component gives equal weight to the user-based and item-based KNN predictions:

$$p_{ij}^{\text{knn}} = 0.50 \hat{c}_{ij}^{\text{user}} + 0.50 \hat{c}_{ij}^{\text{item}}.$$

The value $\gamma = 0.70$ means that the final probability uses 70% weight from the KNN component and 30% weight from the prior component:

$$p_{ij} = 0.70p_{ij}^{\text{knn}} + 0.30p_{ij}^{\text{prior}}.$$

Therefore, after expanding all weights, the final probability gives 35% weight to user-based KNN, 35% weight to item-based KNN, 15% weight to the question prior, and 15% weight to the student prior.

The threshold $\tau = 0.55$ means that the model predicts a correct response when

$$p_{ij} \geq 0.55,$$

and predicts an incorrect response otherwise. This threshold was selected because it gave the highest validation accuracy among the tested values.

Overall, the results support our hypothesis. Combining user- and item-based KNN captures complementary similarity information, while the student and question priors provide lower-variance information that helps stabilize predictions in sparse regions of the response matrix. The fact that the best (γ, τ) combination lies at the edge of the searched grid suggests that a wider search over γ and τ could be explored in future work.

4. Limitations

Sparsity is only partially addressed. The priors help because both students and questions have more observed entries on average than a single KNN neighbourhood does. However, each prior is still only a single scalar for each student or question. For students or questions with very few responses, the prior may be unreliable and may fall back to the global average, which provides little personalization. The approach would also perform poorly for a true cold-start student or question with zero training observations, since both the KNN predictors and the priors would contain very little useful information.

Heuristic rather than principled combination. Unlike IRT, which derives $p(c_{ij} = 1)$ from an explicit probabilistic model with interpretable student-ability and question-difficulty parameters, our hybrid combines KNN outputs and priors using validation-selected linear weights (α, β, γ) and a classification threshold τ . The method does not provide a principled rule for how much weight each component should receive in general. The selected weights are dataset-dependent and would likely need to be re-tuned for a different response matrix or a different student-to-question ratio.

Sensitivity to hyperparameter selection and to the computing environment. The validation set is relatively small, so it may not reliably distinguish between several configurations with very similar accuracies. We also observed, while re-running our own code, that the selected $(\alpha, \beta, \gamma, \tau)$ combination is not perfectly stable across re-runs on different machines. Small floating-point differences in the nearest-neighbour distance computation across library or hardware versions may change which of several near-tied grid points is selected as the arg-max. This means that the reported test accuracy is only a point estimate, and a different validation split or computing environment could select a slightly different configuration. A more robust extension would use cross-validation, a larger held-out set, or an explicit tie-breaking and stability check for the final hyperparameter selection.

Unused structure. We did not use `question_meta.csv` or `student_meta.csv`. A natural extension would be to use a hierarchical prior. For example, if a question has very few observed

responses, its difficulty estimate could be backed off to the average correctness rate of questions from the same subject rather than directly to the global average. Student metadata could also be incorporated to improve predictions for students with limited response histories.

Under-explored search range. The selected (γ, τ) lies at the edge of the searched range, with validation accuracy still increasing toward that boundary rather than reaching a clear interior maximum. This suggests that the current search range may not contain the true best configuration. A follow-up search over a wider range of γ and τ , possibly using finer steps, could determine whether further improvement is possible.